MODULAR BUILD PLAN

ITC-Rec Engine — M0 to M10

Eleven modules, built one at a time, then connected through shared data contracts

The principle that makes this modular: **freeze the data contracts first**, then every module just reads and writes those shapes — so you can build each one against fixtures or stubs and wire them together later through a single orchestrator. The work moves through six phases.

Phases

Phase	Focus	Modules
0	Foundation	M0
1	Pure core (no infrastructure)	M1 · M2
2	Data in	M3 · M4 · M5
3	Wire it together	M6
4	See results	M7 · M8
5	Close the loop	M9 · M10

Build first: M0 → M1 → M2. The schema is the contract everything shares, and the normalizer + matching engine are pure libraries with zero infrastructure — no DB, API, or UI. They carry the core value of the product and are the easiest things to unit-test in isolation. Get them green and the hardest part is de-risked before you write a single endpoint.

Module breakdown

#	Module	What it does	Interface (in → out)	Depends on	Build solo?
M0	Schema & skeleton	Repo, env, migrations for all 10 tables; shared ORM models + typed record shapes	— → DB schema + `Invoice` / `MatchResult` types (the shared contract)	none	Yes
M1	Normalizer (pure lib)	Clean GSTIN, strip invoice-no delimiters / leading zeros, parse dates, standardize tax heads	raw field / row → normalized field / row (pure functions)	none	Yes
M2	Matching engine (pure lib)	4-pass compare (exact → normalized → tolerance → fuzzy); assign bucket, pass, confidence, tax_diff	normalized PR list + 2B list → `MatchResult` list	M1 (shape only)	Yes
M3	Auth & org	Email / password login, session / JWT, single org + current-user context	credentials → session + user context	M0	Partly
M4	Ingestion & parser	Accept upload, detect PR vs 2B, parse CSV / XLSX / JSON, store raw file, write invoice rows (via M1)	file + type + run_id → invoice rows + `uploaded_file`	M0, M1	Yes (parsing)
M5	Audit log (cross-cutting)	Append-only record of every state change	actor + action + entity + details → audit row	M0	Yes
M6	Recon orchestrator	The first big connect: create run → ingest (M4) → match (M2) → persist → update status → log (M5)	run_id (+ files) → `match_result` rows + run status	M0, M2, M4, M5	No (integration)
M7	Results / dashboard API	Bucket counts, per-bucket lists, drill-down, run summary	run_id (+ filters) → JSON summaries / lists	M0, M6 output	Yes (seeded)
M8	Frontend SPA	Create run, upload PR / 2B, trigger recon, show buckets + drill-down	calls M3 / M4 / M6 / M7 → screens	M3, M4, M6, M7	No
M9	Review / force-match	Confirm / reject probable matches, re-bucket, write audit	match_id + decision → updated `match_result` + audit	M0, M6, M5 (UI: M8)	Partly
M10	Report & notice generator	Export reconciliation report (XLSX / PDF), per-vendor discrepancy notices, safe-to-claim list	run_id / match_results → XLSX / PDF files	M0, M6 output	Yes (fixtures)

Build solo? — **Yes**: fully testable in isolation with fixtures/stubs. **Partly**: needs M0 wiring but little else. **No**: an integration step that connects finished modules.

What unlocks “build them one by one”

The two contracts to freeze on day one

Fix these two shapes in M0 and any module can be built and tested against stub data:

- `InvoiceRecord` — the invoice shape produced by ingestion (M4) and consumed by the normalizer (M1) and matcher (M2).
- `MatchResultRecord` — produced by the matcher (M2) and consumed by the orchestrator (M6), dashboard API (M7), review (M9) and report generator (M10).

Fastest path to something testable

Don't wait for the UI. After M0–M2, build a tiny CLI or a single `/reconcile` endpoint that takes two CSVs, runs M1 + M2 (passes 1–2 only), and prints bucket counts. That's a working vertical slice you can throw real data at within a day or two — then layer on M4's file handling, M7's API, and M8's screens.

Order at a glance

1. M0 — schema + skeleton (freeze the two contracts here).
2. M1 — normalizer (pure functions, heavy unit tests).
3. M2 — matching engine, passes 1–2 first, then 3–4.
4. Vertical slice: CLI / `/reconcile` over two CSVs → bucket counts.
5. M3 · M4 · M5 — auth, ingestion/parser, audit log.
6. M6 — orchestrator (the first real connect).
7. M7 · M8 — dashboard API + React SPA.
8. M9 · M10 — review / force-match, then report & notice export.